

TMC-103 – OPERATING SYSTEM

M.C.A. (First Semester) — Mid Semester Examination, Jan. 2023

Complete Solved Answer Key with Detailed Explanations

Reference: *Operating System Concepts*, 9th Edition — Abraham Silberschatz, Peter B. Galvin, Greg Gagne

Note: The original paper requires attempting **one sub-question [(a) OR (b)] from each of the 5 questions**, 10 marks each (Total = 50 marks). For complete revision of the syllabus, **both alternatives of every question are solved in full below**.

Q1. (a) Define OS. Explain the different types of OS with example.

[CO1] Marks: 10

Definition of Operating System

An Operating System (OS) is system software that acts as an intermediary between the user/application programs and the computer hardware. It manages hardware resources (CPU, memory, I/O devices, storage) and provides a convenient environment in which the user can execute programs efficiently. Silberschatz describes the OS as a "resource allocator" that manages resources among competing processes, and as a "control program" that controls the execution of user programs to prevent errors and improper use of the system.

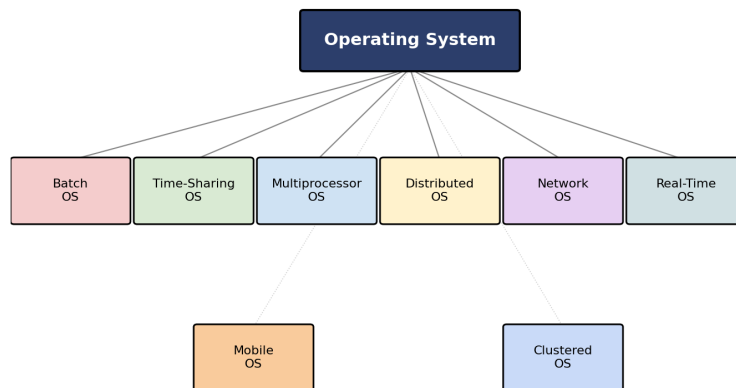


Fig. 1: Classification of Operating Systems

Types of Operating System

1. Batch Operating System

Jobs with similar needs are grouped/batched and executed together without user interaction; an operator or resident monitor loads and runs jobs one after another. Improves CPU throughput but increases turnaround time. **Example:** Early IBM batch systems.

2. Time-Sharing (Multitasking) OS

CPU time is shared among multiple users/programs using time-slicing so each gets fast, interactive response, giving the illusion of simultaneous execution. **Example:** UNIX, Windows, Linux.

3. Multiprocessor OS

Manages a computer system with two or more CPUs sharing the system bus, memory and peripheral devices, providing increased throughput and reliability (graceful degradation on CPU failure). **Example:** Windows Server, Linux with SMP support.

4. Distributed OS

Manages a group of physically separate, networked computers so they appear to users as a single coherent system, allowing resource sharing (files, computation) across machines. **Example:** Amoeba, LOCUS.

5. Network OS

Runs on a server and provides services such as file sharing, printer sharing and user authentication to other computers on a network, while each machine retains its own local autonomy/OS. **Example:** Novell NetWare, Windows Server.

6. Real-Time OS (RTOS)

Provides guaranteed response within strict time constraints for time-critical applications (hard or soft real-time). **Example:** VxWorks, QNX, RTLinux – used in embedded/industrial control.

7. Mobile OS

Designed for smartphones/tablets with touch-based interfaces, limited resources, power management and app sandboxing. **Example:** Android, iOS.

8. Clustered Systems

Two or more individual systems (nodes) are coupled together, sharing storage and connected via a fast local network, to work as a single system providing high availability. **Example:** Microsoft Cluster Server, Oracle RAC.

Q1. (b) Define Kernel. Explain the various types of Kernels with example.

[CO1] Marks: 10

Definition of Kernel

The kernel is the core component of an operating system – the part that is loaded into memory first and remains resident throughout system operation. It runs in privileged (kernel) mode with full access to the hardware, and is responsible for the most fundamental functions: process and memory management, handling interrupts and system calls, and controlling I/O device communication. All other software, including the shell and applications, relies on services provided by the kernel.

Types of Kernels

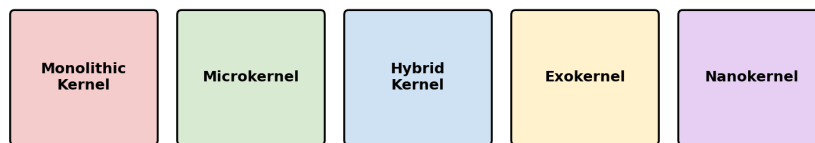


Fig. 2: Types of Kernels

1. Monolithic Kernel

The entire OS – process management, memory management, file systems, device drivers – runs as one single large program in kernel space. Communication between components is via direct function calls, making it fast, but a bug in any part (e.g., a driver) can crash the whole system. **Example:** traditional UNIX, Linux, MS-DOS.

2. Microkernel

Only the most essential services (IPC, minimal scheduling, minimal memory management) run in kernel mode; other services (file systems, device drivers) run as separate user-space servers communicating via message passing. More robust/modular, but message-passing adds overhead. **Example:** Mach, QNX, MINIX.

3. Hybrid Kernel

Combines characteristics of monolithic and microkernels – keeps a small set of core services in kernel space like a microkernel for modularity, but also runs some additional services in kernel space (unlike a pure microkernel) for better performance. **Example:** Windows NT/10/11 kernel, macOS XNU kernel.

4. Exokernel

A minimal kernel that allocates raw hardware resources (CPU cycles, disk blocks, memory pages) securely to library operating systems running in user space, without imposing any particular abstraction, giving applications maximum control and efficiency. **Example:** MIT Exokernel (research OS), Nemesis.

5. Nanokernel

An extremely minimal kernel providing only the most basic hardware abstraction (often just interrupt handling), delegating virtually everything else, including device drivers, to higher-level software. **Example:** KeyKOS, L4 microkernel family (in its nanokernel form).

Q2. (a) Draw and explain the various states of a process.

[CO1/CO2] Marks: 10

As a process executes, it changes state depending on its current activity. Each process may be in one of the following five states at any instant of time (Silberschatz, Ch.3):

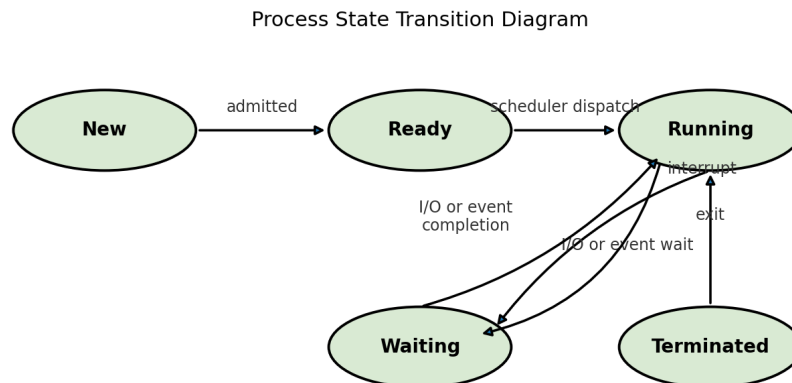


Fig. 3: Process State Transition Diagram

- **New** – the process is being created; the OS is performing admission tasks like assigning a PID and initializing the PCB.
- **Ready** – the process has all resources it needs except the CPU, and is waiting to be assigned to a processor by the scheduler.
- **Running** – the process's instructions are actually being executed by the CPU.
- **Waiting (Blocked)** – the process is waiting for some event to occur, e.g. completion of I/O or a signal from another process.
- **Terminated** – the process has finished execution (or was aborted) and is being removed from the system.

State Transitions

- **New → Ready** – process admitted by the long-term scheduler.
- **Ready → Running** – process is dispatched/selected by the short-term (CPU) scheduler.
- **Running → Waiting** – process issues an I/O request or waits for an event/resource.
- **Waiting → Ready** – the event the process was waiting for has occurred.
- **Running → Ready** – an interrupt occurs (e.g., time quantum expires) and the process is preempted.
- **Running → Terminated** – the process completes execution or is aborted by the OS/parent.

At any given time, only one process can be in the running state on a single CPU core, though many processes may simultaneously be in the ready or waiting states. The OS keeps track of the current state of every process using its Process Control Block (PCB).

Q2. (b) Differentiate between the following: (i) Multiprogramming vs. Multitasking OS (ii) System call vs. Functions

[CO1/CO2] Marks: 10 (5 marks each)

(i) Multiprogramming OS vs Multitasking OS

Basis	Multiprogramming OS	Multitasking OS
Concept	Several jobs reside in main memory simultaneously and the CPU always has a CPU task to execute. It switches between jobs.	Execution of the CPU always has a CPU task to execute. It switches between jobs.
Primary Goal	Maximize CPU utilization by minimizing idle time.	Provide fast, interactive response time to the user.
Switching trigger	CPU switches to another job mainly when the CPU switches requests I/O.	CPU switches to another job mainly when the CPU switches requests I/O.
User interaction	Little/no direct interaction; jobs mostly run in batch mode.	Highly interactive; user perceives simultaneous execution of jobs.
Example	Old mainframe batch-cum-multiprogrammed systems.	Modern desktop/laptop OS such as Windows, Linux running on modern PCs.

(ii) System Call vs Functions

Basis	System Call	Function (Library/User function)
Definition	A request made by a user program to the OS kernel to perform a privileged service.	A self-contained block of code performing a specific task, written by a programmer.
Execution mode	Causes a switch from user mode to kernel mode.	Executes entirely in user mode within the calling process's own address space.
Who provides it	Provided by the operating system kernel.	Provided by the programmer or a user-level library (e.g., C standard library).
Overhead	Higher – involves mode switching and context switch.	Lower – simple procedure call/return within the same process.
Purpose	Access hardware/kernel resources: file I/O, process creation, etc.	Perform general-purpose computation reusable within the program.
Example	open(), fork(), read(), write(), exit().	A user-defined function add(a,b) or a library call like printf() (which is a system call).

Q3. (a) Draw and explain the PCB. Also explain the PCB in memory.

[CO2] Marks: 10

Process Control Block (PCB)

Every process in an operating system is represented by a data structure in the kernel called the Process Control Block (also called a Task Control Block). It stores all the information the OS needs to manage the process, track its status, and later correctly resume it after an interrupt or context switch.

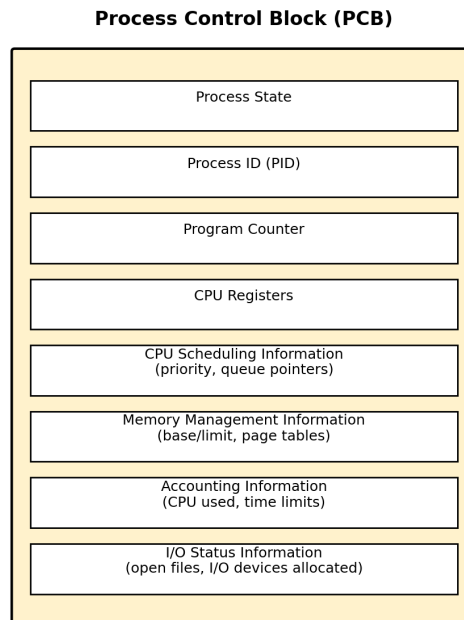


Fig. 4: Structure of a Process Control Block (PCB)

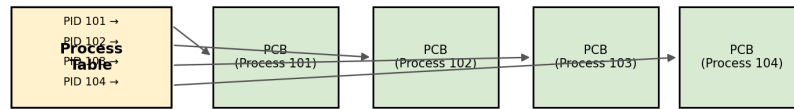
Fields of a PCB

- **Process state** – new, ready, running, waiting or terminated.
- **Process ID (PID)** – unique identifier for the process, and a pointer to its parent process.
- **Program counter** – address of the next instruction to be executed for this process.
- **CPU registers** – accumulators, index registers, stack pointers, general-purpose registers, saved during an interrupt so execution can resume correctly.
- **CPU scheduling information** – process priority, pointers to scheduling queues, other scheduling parameters.
- **Memory management information** – base/limit register values, page tables or segment tables.
- **Accounting information** – amount of CPU/real time used, time limits, process numbers.
- **I/O status information** – list of I/O devices allocated to the process, list of open files.

PCB in Memory

PCBs are stored by the operating system in a protected area of kernel memory called the **process table** (an array or linked list of pointers, one per active process). Because this area is accessible only in kernel mode, a user process cannot directly read or corrupt another process's PCB, which is essential for protection.

Process Table (in OS kernel memory) — array/list of pointers to PCBs



Each PCB physically resides in protected kernel-memory space (not user space) so that a user process cannot directly read or modify another process's control information. The scheduler manipulates PCBs (not the process image) by moving their pointers between the ready queue, device queues, etc., which is far cheaper than moving the process itself.

Fig. 5: PCBs organised via the process table in kernel memory

Whenever the scheduler needs to move a process between queues (e.g., from ready queue to running, or running to a device's waiting queue), it does so simply by relinking the process's PCB pointer into the appropriate queue – the large process image itself does not need to be moved. During a **context switch**, the OS saves the CPU state of the currently running process into its PCB, and loads the previously saved CPU state of the next scheduled process from its PCB into the CPU registers, allowing that process to resume exactly where it left off.

Q3. (b) Differentiate between the following: (i) Process vs. Threads (ii) Open-source vs. Commercial OS

[CO1] Marks: 10 (5 marks each)

(i) Process vs Threads

Basis	Process	Thread
Definition	A program in execution; an independent unit with its own memory space.	A lightweight unit of execution within a process; also called a sub-process.
Memory	Has its own separate address space (code, data, heap, stack).	Shares the address space of the process it belongs to.
Creation overhead	Heavyweight – costly to create/terminate (high kernel involvement).	Lightweight – fast to create/terminate, fewer resource requirements.
Communication	Requires inter-process communication (IPC) mechanisms like pipes, shared memory, message queues.	Communicates via shared memory or message passing within the process.
Independence	Independent; a crash in one process generally does not affect another.	Not independent; one thread can bring down the whole process.
Example	Running a browser and a word processor are two separate processes.	A UI-rendering thread within the same browser process.

(ii) Open-source OS vs Commercial OS

Basis	Open-source OS	Commercial (Proprietary) OS
Source code	Freely available for anyone to view, modify and distribute.	Closed and proprietary; source code is not disclosed to users.
Cost	Usually free of cost (though paid support may be available).	Sold under a paid license.
Customization	Highly customizable by developers/community.	Limited or no customization allowed by end users.
Support & updates	Community-driven support; updates depend on community efforts.	Official vendor support (sometimes for a fee/paid support).
Security auditing	Code can be publicly reviewed, so vulnerabilities are often found quickly.	Security relies on vendor's internal checks; harder to audit independently.
Example	Linux (Ubuntu, Fedora), FreeBSD.	Microsoft Windows, Apple macOS/iOS.

Q4. (a) Explain different types of schedulers and various queues.

[CO1/CO2] Marks: 10

A process migrates between various scheduling queues throughout its lifetime, and the OS selects processes from these queues using different schedulers.

Types of Schedulers

- **Long-term Scheduler (Job Scheduler)** – selects which jobs/processes from the job pool (on disk) should be brought into main memory for execution; controls the *degree of multiprogramming*. It runs infrequently (seconds/minutes) and can afford to be slower; it aims to keep a good mix of CPU-bound and I/O-bound processes.
- **Short-term Scheduler (CPU Scheduler)** – selects, from the ready queue, which process the CPU should execute next. It runs very frequently (every few milliseconds), so it must be extremely fast.
- **Medium-term Scheduler** – used in time-sharing systems to temporarily swap a process out of main memory (to reduce the degree of multiprogramming) and swap it back in later, improving the process mix or freeing up memory when needed.

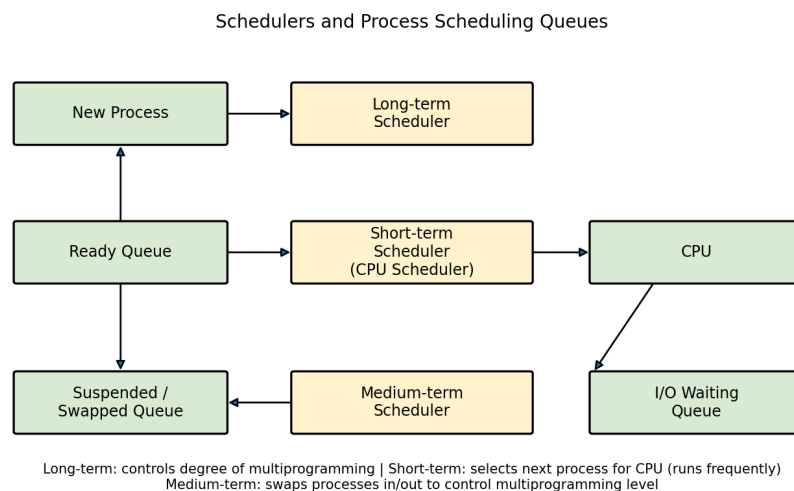


Fig. 6: Schedulers and process scheduling queues

Scheduling Queues

- **Job Queue** – contains all processes in the system, residing on the mass-storage device, waiting to be admitted into main memory.
- **Ready Queue** – contains all processes residing in main memory that are ready and waiting to execute; usually maintained as a linked list of PCBs.
- **Device (I/O) Queues** – each I/O device has its own queue; processes waiting for a specific device are placed there. A process may move between the ready queue and several device queues multiple times during its life.

A newly created process joins the ready queue and waits to be dispatched. Once running it may (a) issue an I/O request and move to a device queue, (b) create a child process and wait for its termination, or (c) be preempted by an interrupt and returned to the ready queue. This continues until the process terminates, at which point it is removed from all queues and its resources (including the PCB) are deallocated.

Q4. (b) Calculate Average waiting time, Average TAT, throughput, CPU utilization for the following using FCFS scheduling Algorithm.

[CO1/CO2] Marks: 10

Process	Arrival Time	CPU Burst Time (ms)
P0	3	2
P1	2	4
P2	0	6
P3	1	4

Concept: First-Come-First-Served (FCFS) is a non-preemptive scheduling algorithm in which processes are executed strictly in the order they arrive in the ready queue.

Step 1: Sort by arrival time and determine execution order

- P2 arrives first at $t=0 \rightarrow$ runs 0–6.
- P3 arrives at $t=1$ (waits till 6) $\rightarrow$ runs 6–10.
- P1 arrives at $t=2$ (waits till 10) $\rightarrow$ runs 10–14.
- P0 arrives at $t=3$ (waits till 14) $\rightarrow$ runs 14–16.

Gantt Chart — FCFS Scheduling

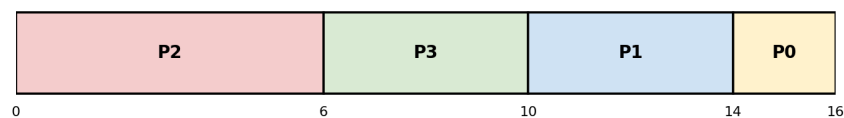


Fig. 7: Gantt chart for FCFS scheduling

Step 2: Completion, Turnaround (TAT) and Waiting Time

$TAT = Completion\ Time - Arrival\ Time$ | $WT = Turnaround\ Time - Burst\ Time$

Process	Arrival	Burst	Completion	Turnaround (CT-AT)	Waiting (TAT-BT)
P2	0	6	6	$6-0 = 6$	$6-6 = 0$
P3	1	4	10	$10-1 = 9$	$9-4 = 5$
P1	2	4	14	$14-2 = 12$	$12-4 = 8$
P0	3	2	16	$16-3 = 13$	$13-2 = 11$

Step 3: Averages

- Average Turnaround Time = $(6+9+12+13)/4 = 40/4 = 10\ ms$
- Average Waiting Time = $(0+5+8+11)/4 = 24/4 = 6\ ms$

Step 4: CPU Utilization

Total elapsed time = 16 ms; total burst time = $6+4+4+2 = 16\ ms$. Since P2 starts exactly at $t=0$ and the CPU is never idle thereafter (each next process has already arrived by the time the CPU frees up), CPU Utilization = $(16/16) \times 100 = 100\%$.

Step 5: Throughput

Throughput = Number of processes completed / Total time = $4/16 = 0.25\ processes\ per\ ms$ (1 process every 4 ms).

Q5. (a) Draw and explain the various services of OS.

[CO1/CO2] Marks: 10

An operating system provides an environment for the execution of programs, offering a set of services both to the programs it runs and to the users who use those programs. These services (Silberschatz, Ch.2) are designed to make computing more convenient and efficient for the user.

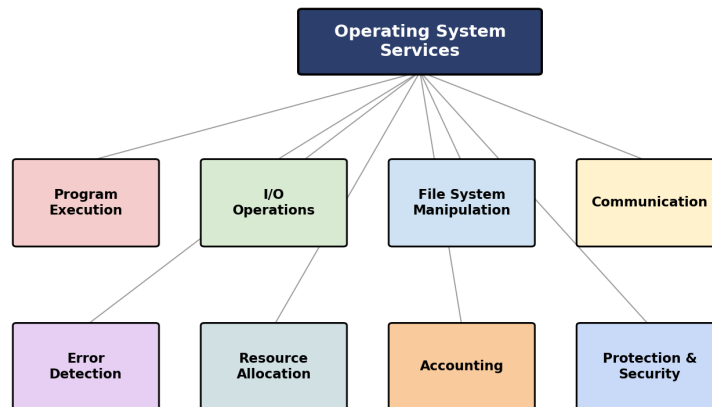


Fig. 8: Services provided by an Operating System

- **Program Execution** – the OS must load a program into memory, run it, and be able to end its execution, either normally or abnormally (indicating an error).
- **I/O Operations** – since user programs cannot directly execute I/O operations for efficiency and protection reasons, the OS provides controlled access to I/O devices on behalf of running programs.
- **File System Manipulation** – programs need to read, write, create and delete files/directories, search for files, list directory information and manage permissions; the OS manages all this via the file system.
- **Communication** – processes need to exchange information with each other, either on the same computer (via shared memory) or over a network (via message passing); the OS implements these mechanisms.
- **Error Detection** – the OS must continuously detect and, where possible, correct errors arising in the CPU, memory hardware, I/O devices, or in user programs, and take appropriate action to ensure correct and consistent computing.
- **Resource Allocation** – when multiple users or multiple jobs are running concurrently, resources such as CPU time, main memory, file storage and I/O devices must be allocated to each of them; the OS manages this via schedulers and allocators.
- **Accounting** – the OS keeps track of which users use how much and what kinds of computer resources, useful for billing or usage statistics.
- **Protection and Security** – in a multiuser/networked environment, the OS must ensure that all access to system resources is controlled, protecting owners of information from unauthorized access and defending the system from external/internal attacks.

Collectively, these services allow the operating system to fulfil its dual role of providing a convenient interface for users and programs while ensuring the efficient, safe and fair use of the underlying hardware resources.

Q5. (b) Calculate Average waiting time, Average TAT, throughput, CPU utilization for non-preemptive SJF scheduling for the following:

[CO1/CO2] Marks: 10

Process	Burst Time (ms)
P1	6
P2	8
P3	7
P4	3

As no arrival times are given, all processes are assumed to arrive at time 0.

Concept: Non-preemptive Shortest Job First (SJF) – among the processes available (here, all at $t=0$), the process with the smallest CPU burst time is executed first, and once started it runs to completion without interruption.

Step 1: Order by burst time (ascending)

P4 (3) → P1 (6) → P3 (7) → P2 (8)

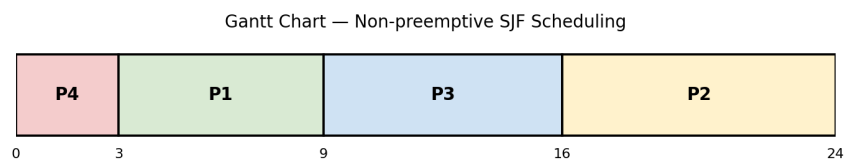


Fig. 9: Gantt chart for non-preemptive SJF scheduling

Process	Burst	Completion	Turnaround (CT-0)	Waiting (TAT-BT)
P4	3	3	3	0
P1	6	9	9	3
P3	7	16	16	9
P2	8	24	24	16

Step 2: Averages

- Average Turnaround Time = $(3+9+16+24)/4 = 52/4 = 13$ ms
- Average Waiting Time = $(0+3+9+16)/4 = 28/4 = 7$ ms

Step 3: CPU Utilization

Total elapsed time = 24 ms; total burst time = $3+6+7+8 = 24$ ms → CPU never idle → CPU Utilization = $(24/24) \times 100 = 100\%$.

Step 4: Throughput

Throughput = Number of processes completed / Total time = $4/24 = 0.1667$ processes per ms (≈ 1 process every 6 ms).